Boring but important disclaimers:

- If you are not getting this from the GitHub repository or the associated Canvas page (e.g. CourseHero, Chegg etc.), you are probably getting the substandard version of these slides Don't pay money for those, because you can get the most updated version for free at

  `https://github.com/julianmak/OCES4303_ML_ocean`

  The repository principally contains the compiled products rather than the source for size reasons.
- Associated Python code (as Jupyter notebooks mostly) will be held on the same repository. The source data however might be big, so I am going to be naughty and possibly just refer you to where you might get the data if that is the case (e.g. JRA-55 data). I know I should make properly reproducible binders etc., but I didn't...
- I do not claim the compiled products and/or code are completely mistake free (e.g. I know I don't write Pythonic code). Use the material however you like, but use it at your own risk.
- As said on the repository, I have tried to honestly use content that is self made, open source or explicitly open for fair use, and citations should be there. If however you are the copyright holder and you want the material taken down, please flag up the issue accordingly and I will happily try and swap out the relevant material.

OCES 4303 :

an introduction to data-driven and ML methods in ocean sciences

Session 10: more neural networks

Outline

- auto-encoders

  → latent space representation

  → de-noising example

- Recurrent Neural Networks (RNNs)

  → hidden states

  → time-series prediction

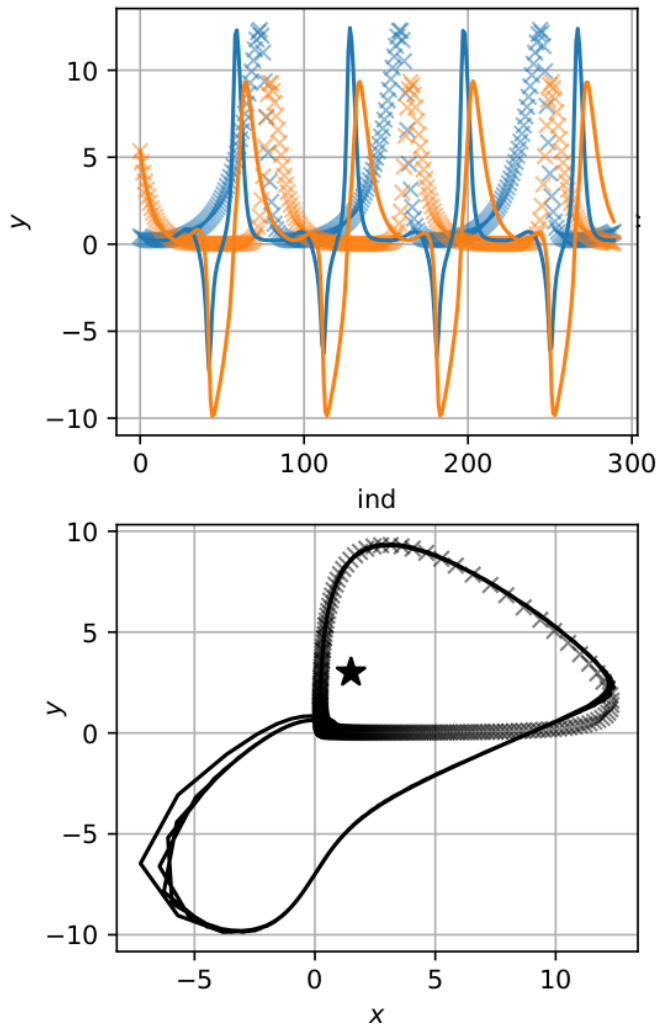

Figure: RNN doing insane things...

Oceanic application

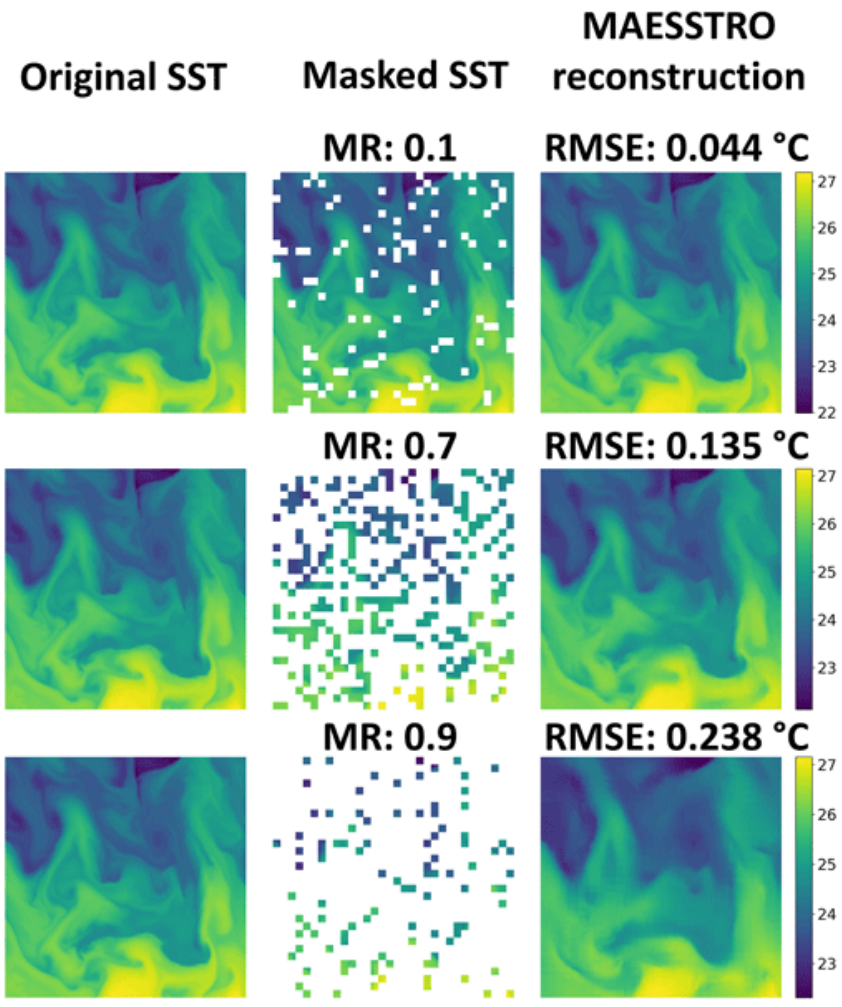

Figure: Reconstruction of SST from occluded data. From Fig. 2 of Goh *et al.* (2024).

- e.g. satellite data

  → masked auto-encoder to fill out the gaps

  → model data but masked accordingly (e.g. cover by cloud)

  → don't have to wait for the satellite to pass again

  → could envisage this being applied to other sparse data (e.g. oxygen)

Auto-encoders

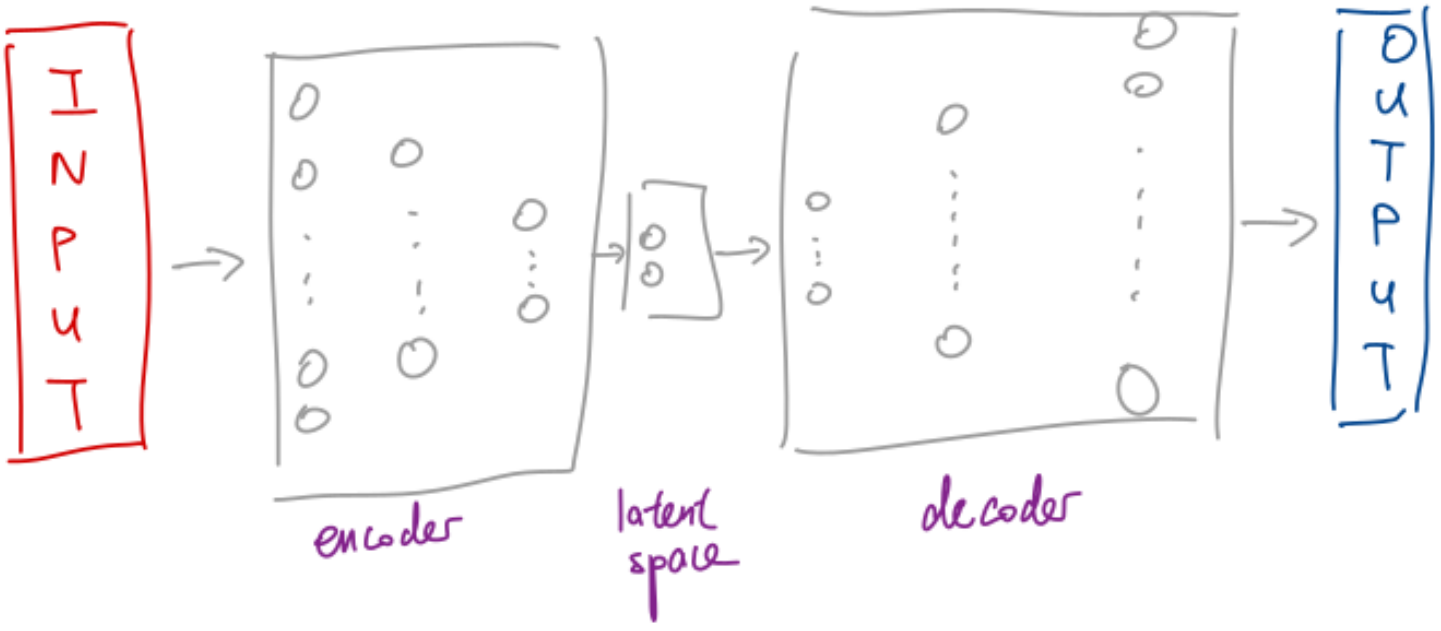

Figure: Schematic of an auto-encoder.

- "unsupervised" neural network

  → encoder part to compress data as latent space representation

  → decoder part to decompress data

Auto-encoders: penguins

- penguins data

  → encoder takes four feature dim to two

  → decoder takes two latent space dim to four

  → input and outputs are exactly the same (hence **auto**-encoder)

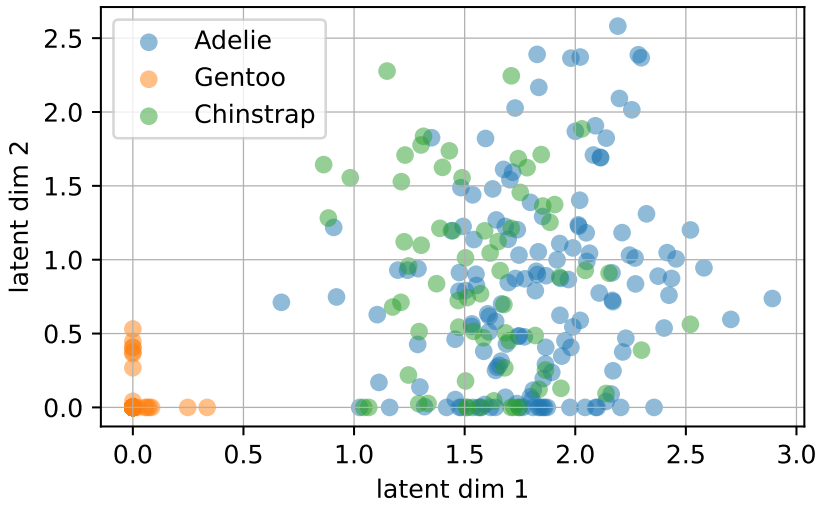

Figure: Latent space representation of penguin data.

- call the encoder only to get latent space representation

  → bit like getting the PCs

  → alternative way to do dimension reduction

Auto-encoders: cats

- do the same with the cat images (sigmoid activation, MinMax scaling)
  $\rightarrow 64^2 \rightarrow 16^2 \rightarrow 4^2$ and reverse
  $\rightarrow$ note the imperfect reconstruction here

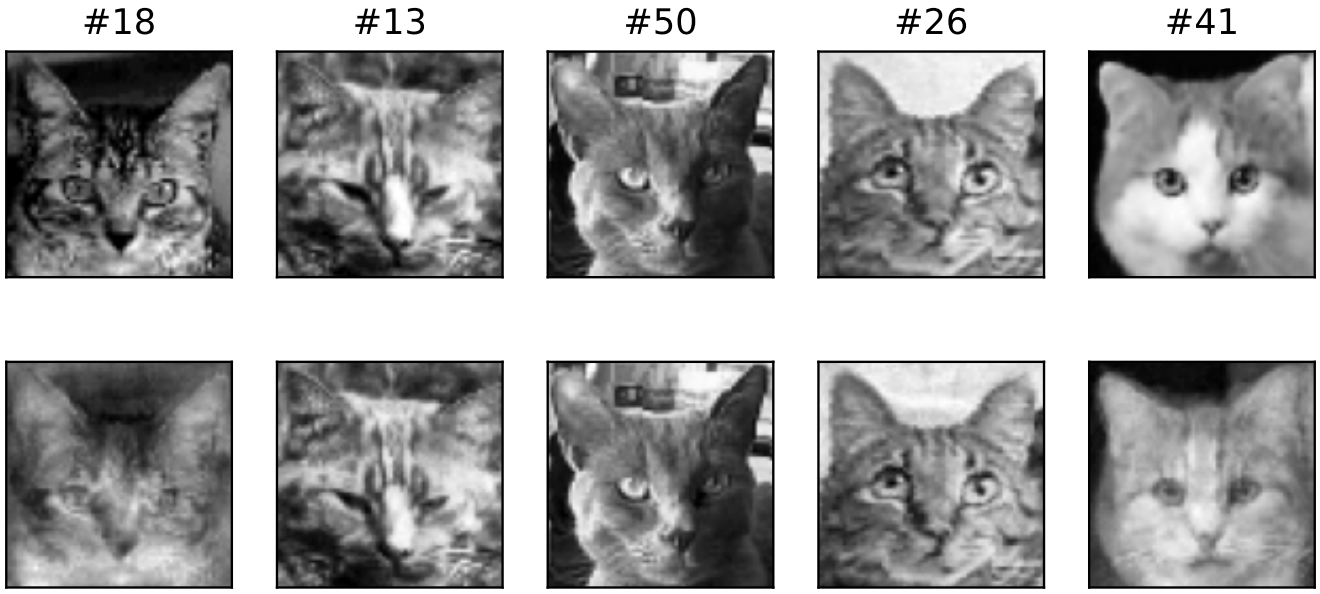

Figure: Training cat images for auto-encoder.

Auto-encoders: cats

- do the same with the cat images

  $\rightarrow$ encoder is $\rightarrow 64^2 \rightarrow 16^2 \rightarrow 4^2$

  $\rightarrow$ notice each cat has a few zero entries

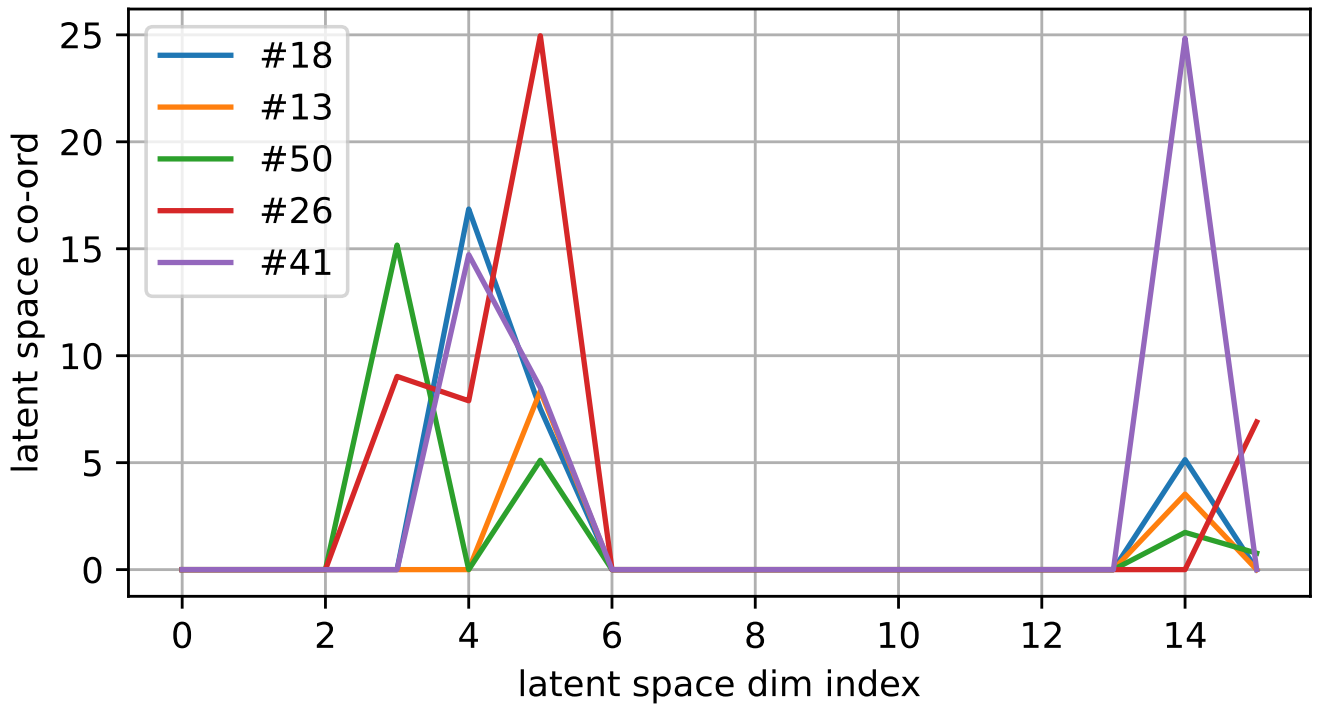

Figure: 16 dimensional latent space representation of the cats in the previous slide (cf. PCs of an EOF decomposition as we did in eigencat).

Auto-encoders: cursed cats

- could generate new cat images by calling the decoder
  $\rightarrow$ decoder is $4^2 \rightarrow 16^2 \rightarrow 64^2$
  $\rightarrow$ choose 16 numbers to feed into the decoder

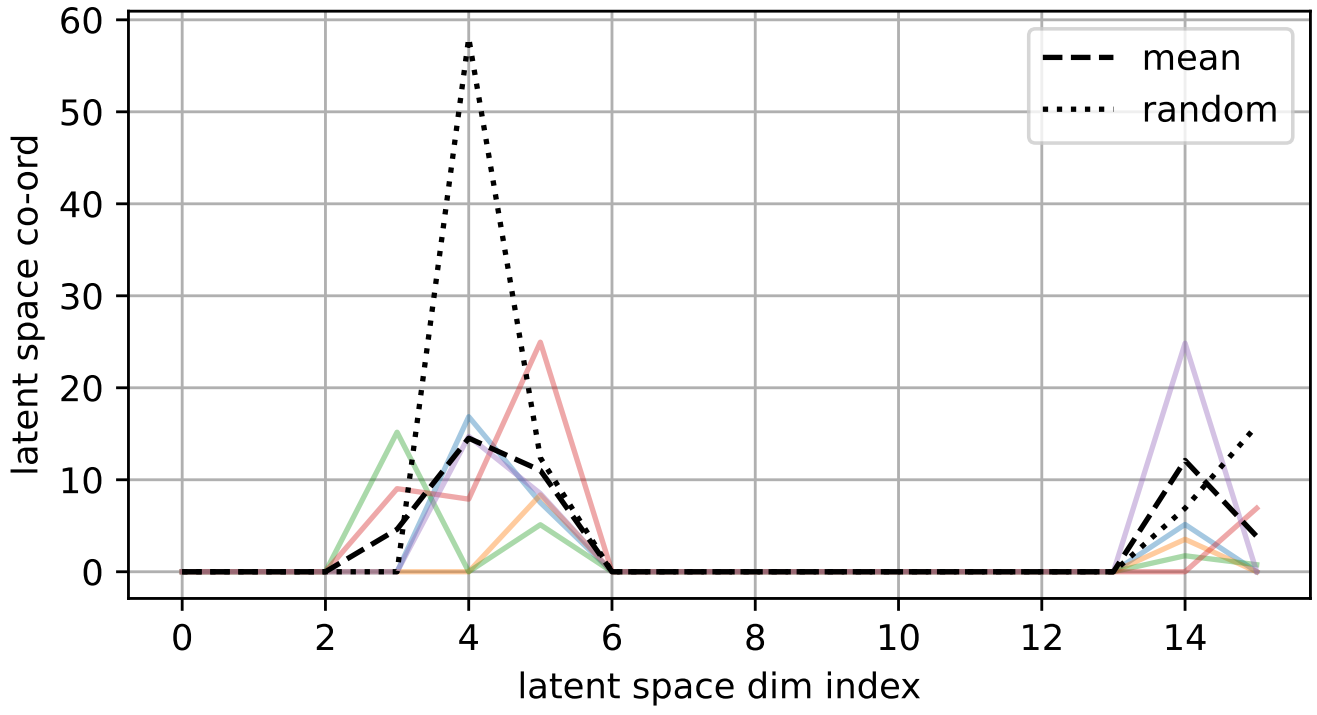

Figure: Two realisations in the 16 dimensional latent space representation to be fed into the decoder to generate new images (cf. choosing PCs for the EOFs and recombine to get image).

Auto-encoders: cursed cats

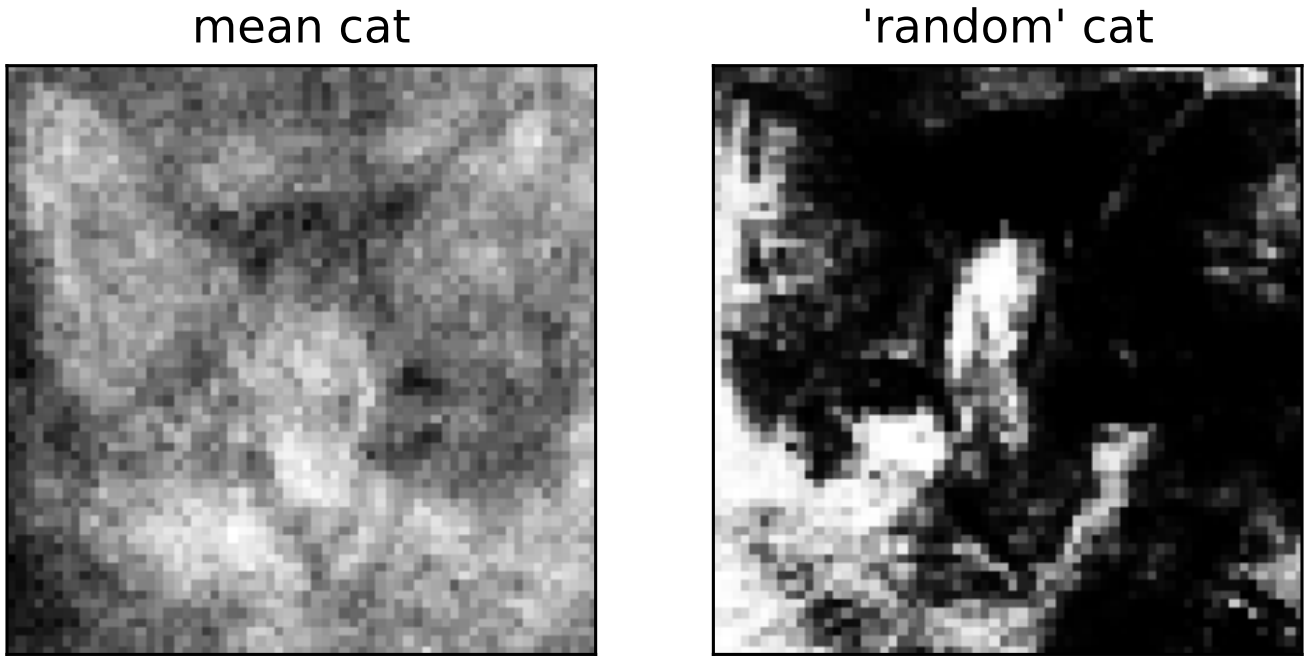

Figure: Slightly cursed regenerations.

- I never said they were going to be good reconstructions...
  $\rightarrow$ small dataset and auto-encoder shallow and thin

Auto-encoders: de-noised cats

- could use auto-encoder to de-noise data
  - $\rightarrow$ training input is noisy data, training output is clean data
  - $\rightarrow$ neural network to learn to remove noise

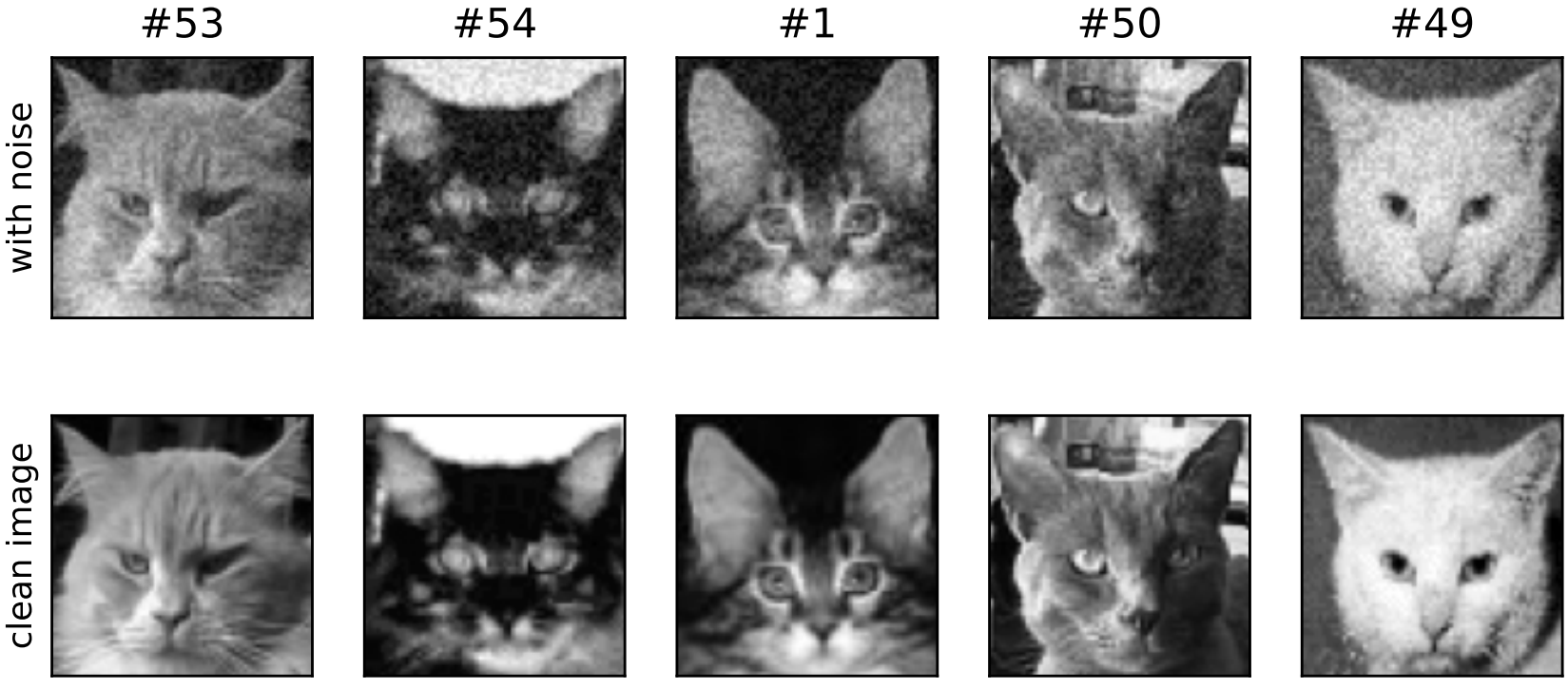

Figure: Noisy cats. Noise level is 0.5 standard deviations (images are standarised).

Auto-encoders: de-noised cats

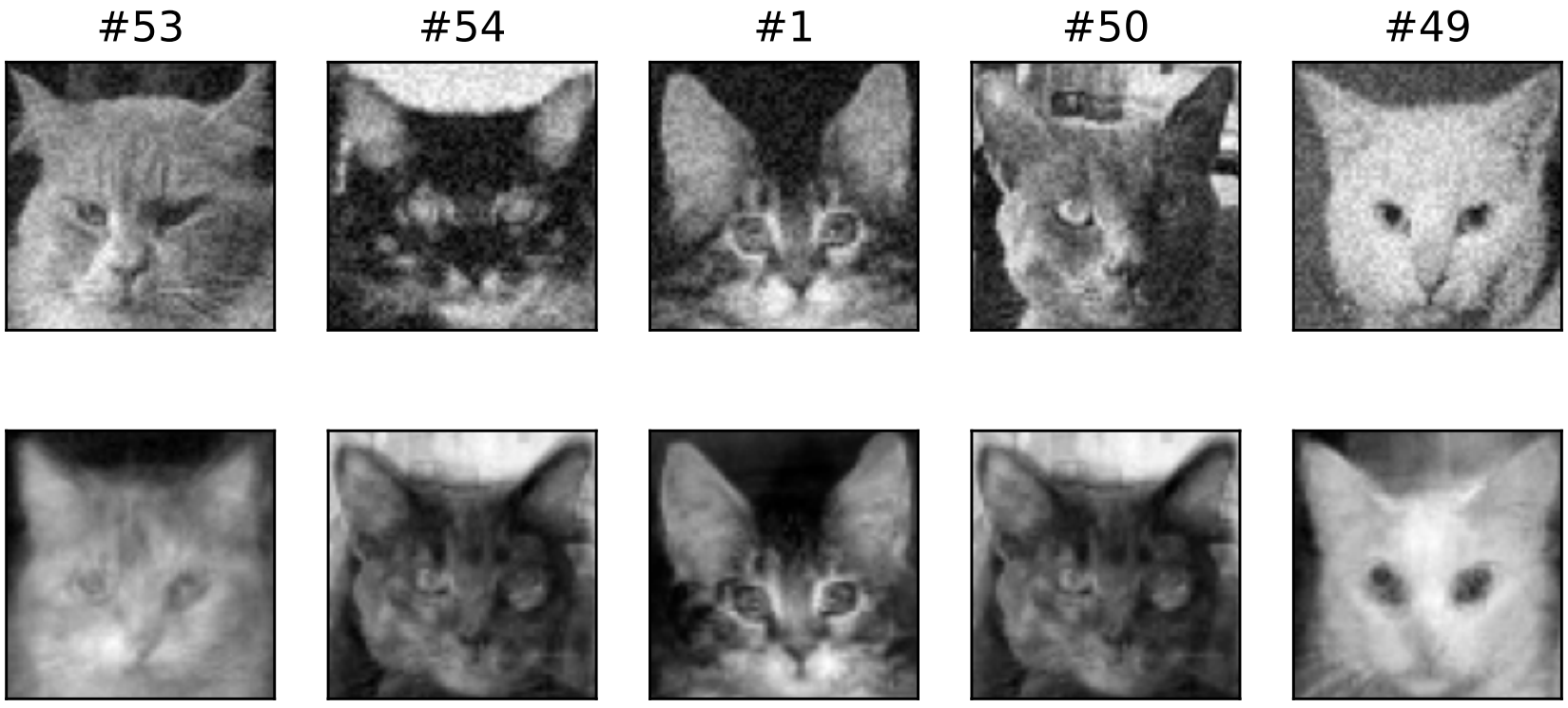

Figure: Training set seems ok...

Auto-encoders: de-noised cats

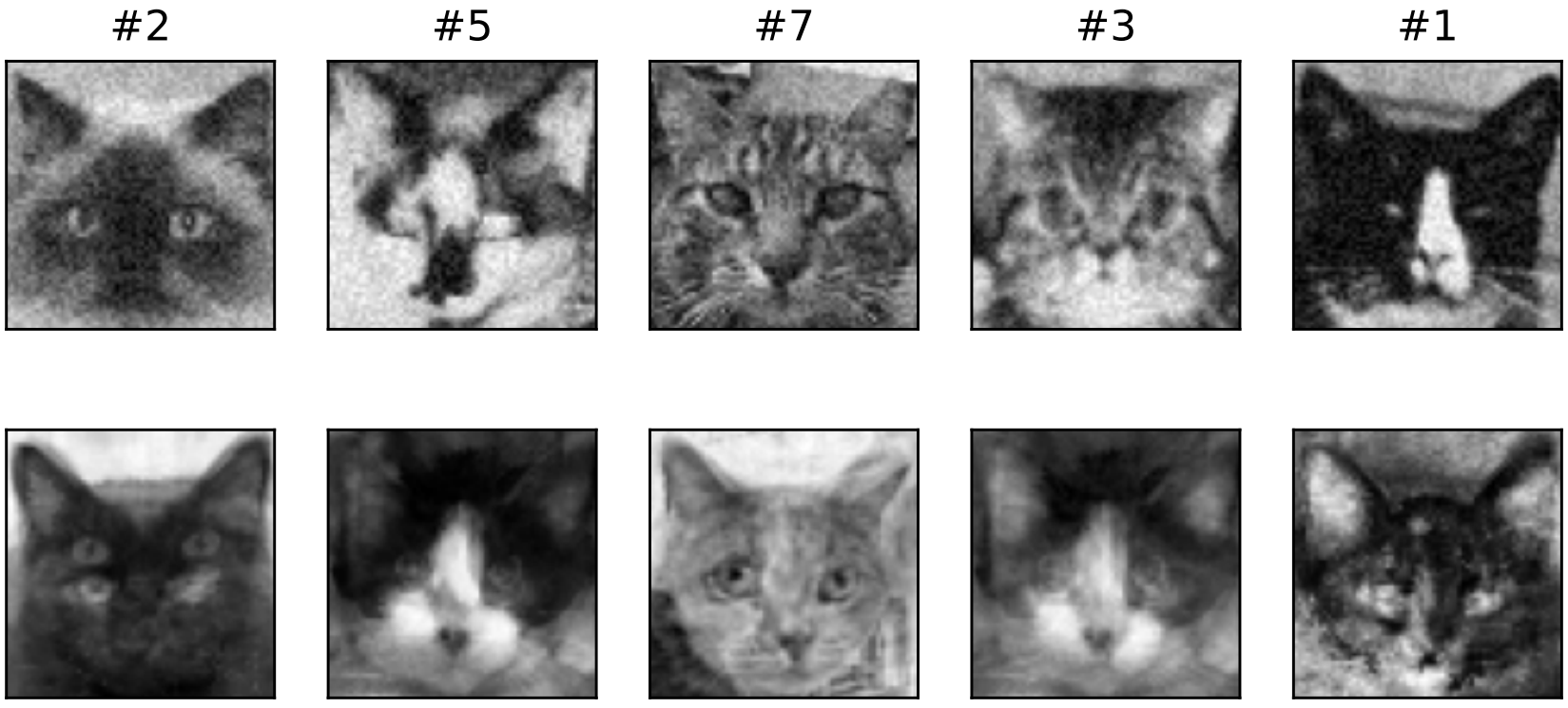

Figure: Testing set is ehhhh...(sample size definitely too small here)

Auto-encoders: de-noised cats

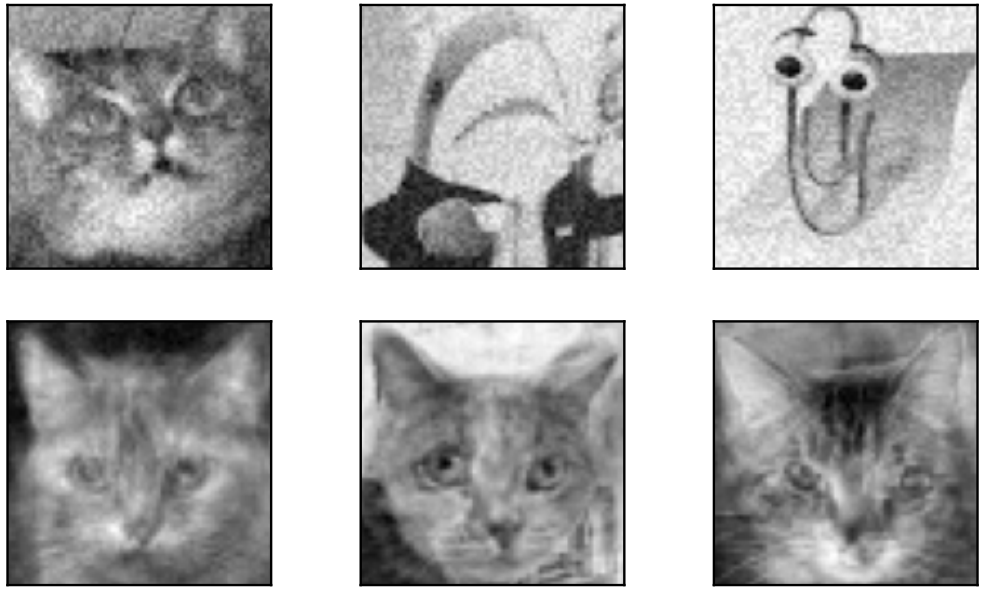

Figure: Just for fun out-of-sample application.

- use larger datasets (e.g. the big eigencat one from lec 03)
- deeper + wider neural network blocks for encoder/decoder
- **convolutional** auto-encoders may work better here also

RNNs

- for **sequences** of data
  → predictive text
  → acoustic data
  → speech data (e.g. translation)
  → **time-series** data (e.g. lec 08 in OCES 3301)

- Recurrent Neural Network (RNNs)
  → some sort of **memory** effect desired

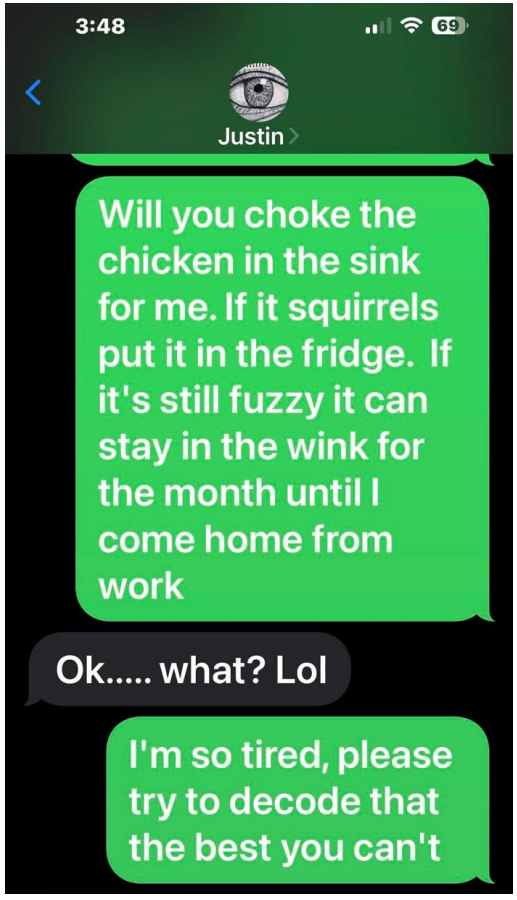

Figure: le wat?

RNNs

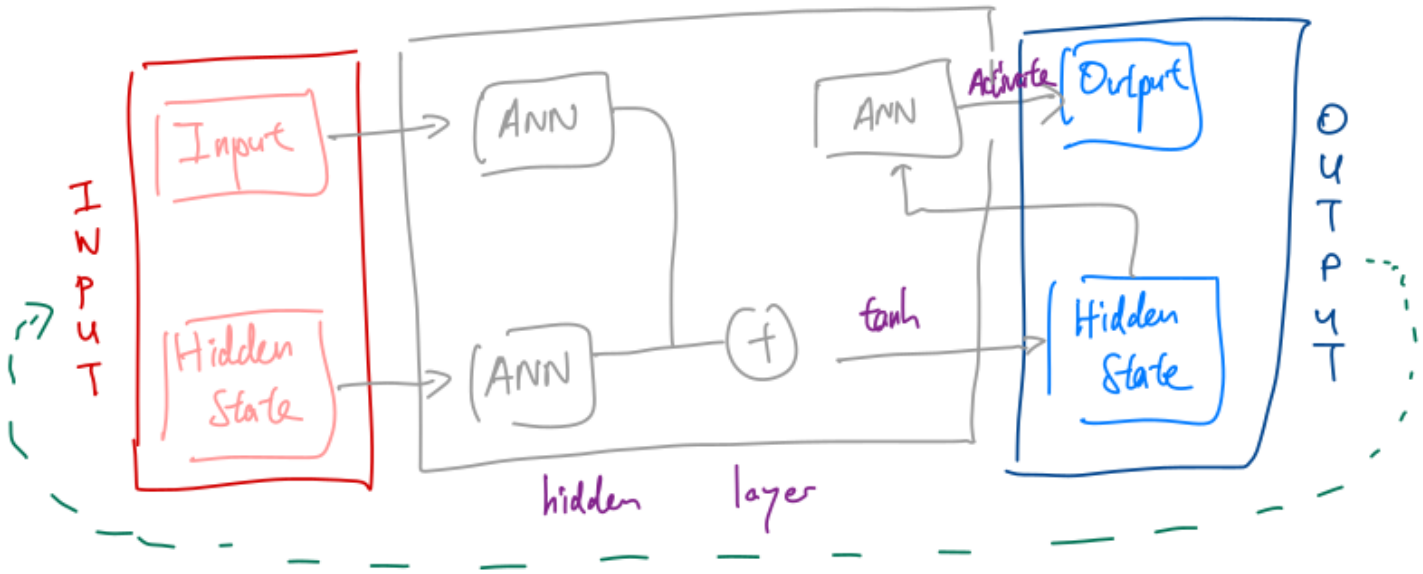

Figure: Demonstrative schematic of a RNN.

- introduction of a hidden state
  → part of the input/output
  → prediction depends on value(s) of hidden state
- for this schematic we have **three** neural networks blocks that are trained

RNNs: Lotka-Volterra

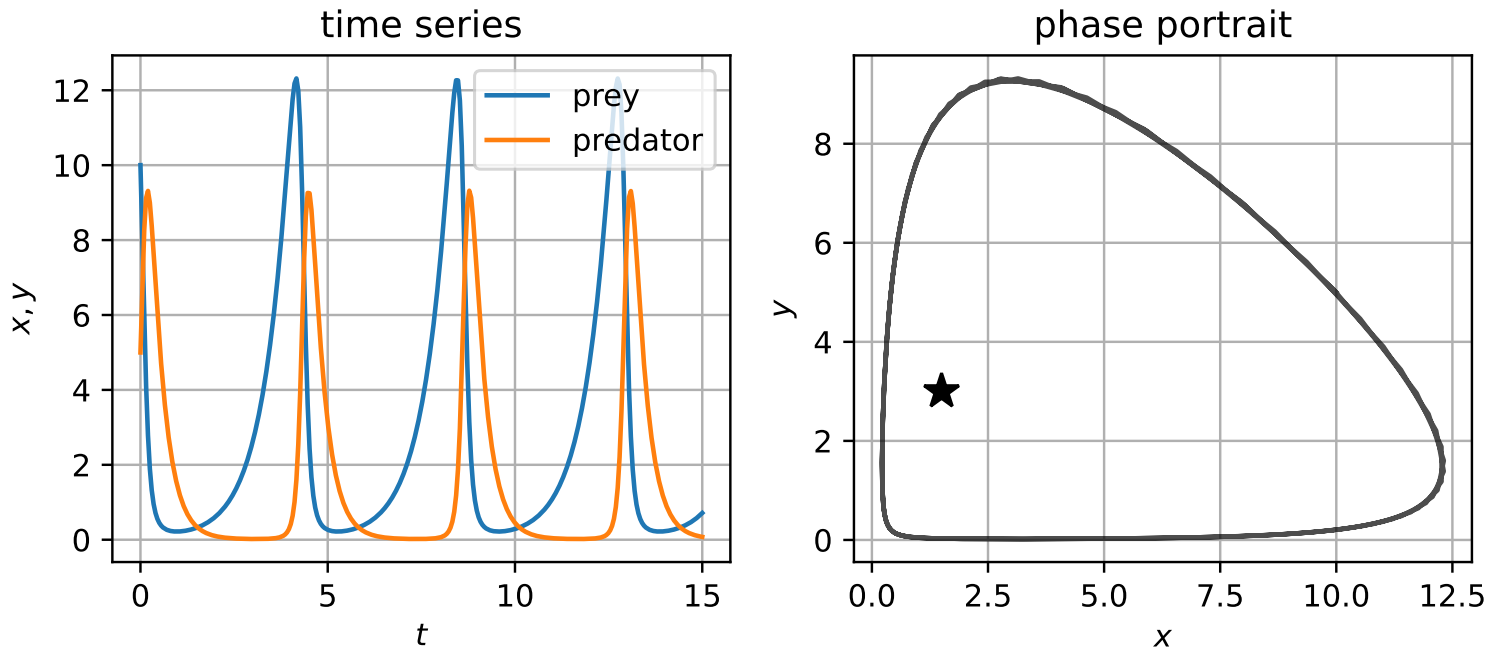

Figure: Time-series from Lotka-Volterra equation as time-series and **phase portrait**.

- artificially generate some data
  → Lotka-Volterra or predator-prey model here
- need to define inputs and outputs
  → inputs are **sequences** of two numbers (x, y)
  → outputs are two numbers (x, y)

RNNs: Lotka-Volterra

- use default RNN in pytorch (Elman RNNs)

  $\rightarrow$ can make these more complex if you write them yourself...

```python
# use torch RNN
class BasicRNN(nn.Module):
    def __init__(self, input_size=1, output_size=1, hidden_size=60, num_layers=1):
        super(BasicRNN, self).__init__()
        self.rnn = nn.RNN(input_size,
                          hidden_size,
                          num_layers,
                          batch_first=True)  # have dimension of batch at position 0
        self.fc = nn.Linear(hidden_size, output_size)

    def forward(self, x):
        out, _ = self.rnn(x)          # RNN output for ALL time steps (don't need hidden state)
                                      # (hidden state defaults to zero is not initialised)
        out = out[:, -1, :]           # output from the last time step

        return self.fc(out)           # Pass through linear layer
```

Figure: Basic RNN from pytorch (Elman RNNs). Note the output suppression of the hidden state in the second argument (we don't really need it).

RNNs: Lotka-Volterra

- train RNN to predict (x_{i+1}, y_{i+1}) from (x_i, y_i)
  $\rightarrow$ one-step prediction

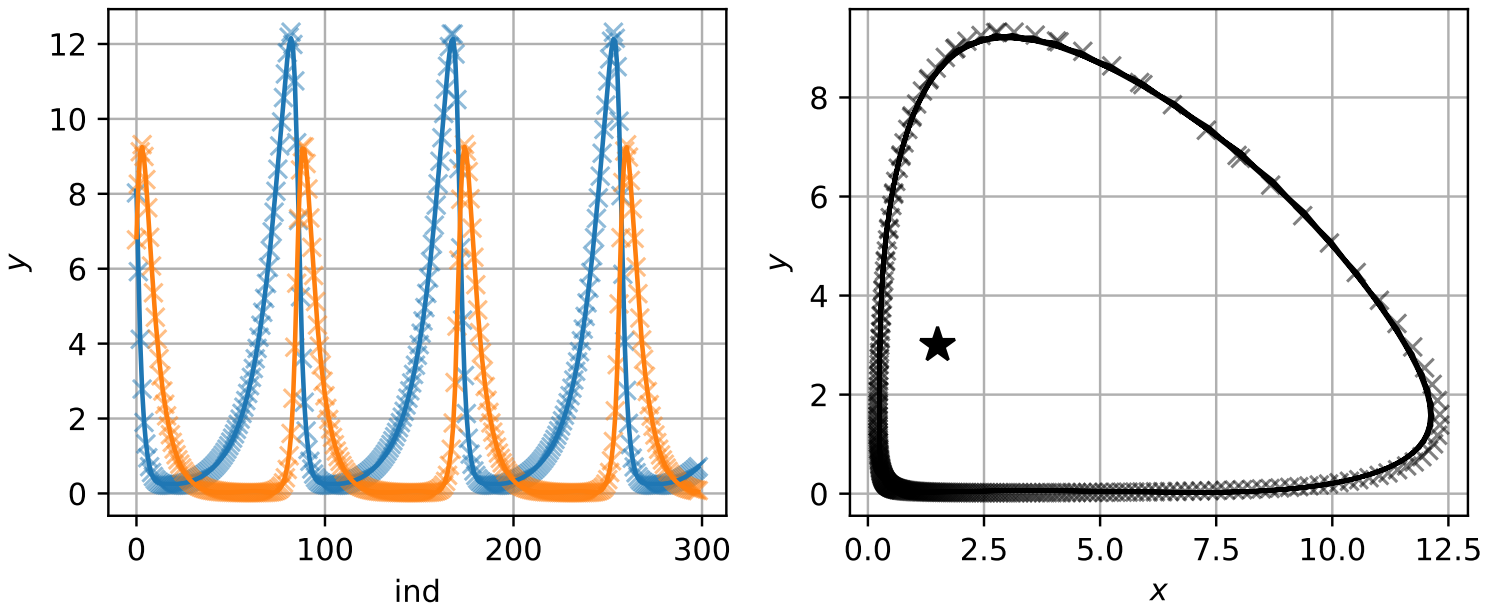

Figure: Easy one-step test. Actual data are markers, and RNN predictions are given as lines.

RNNs: Lotka-Volterra

- harder test: provide initial condition and keep applying the RNN

  $\rightarrow$ errors can (and will) accumulate

  $\rightarrow$ more relevant application, since that's what we want to use the RNN for in some ways

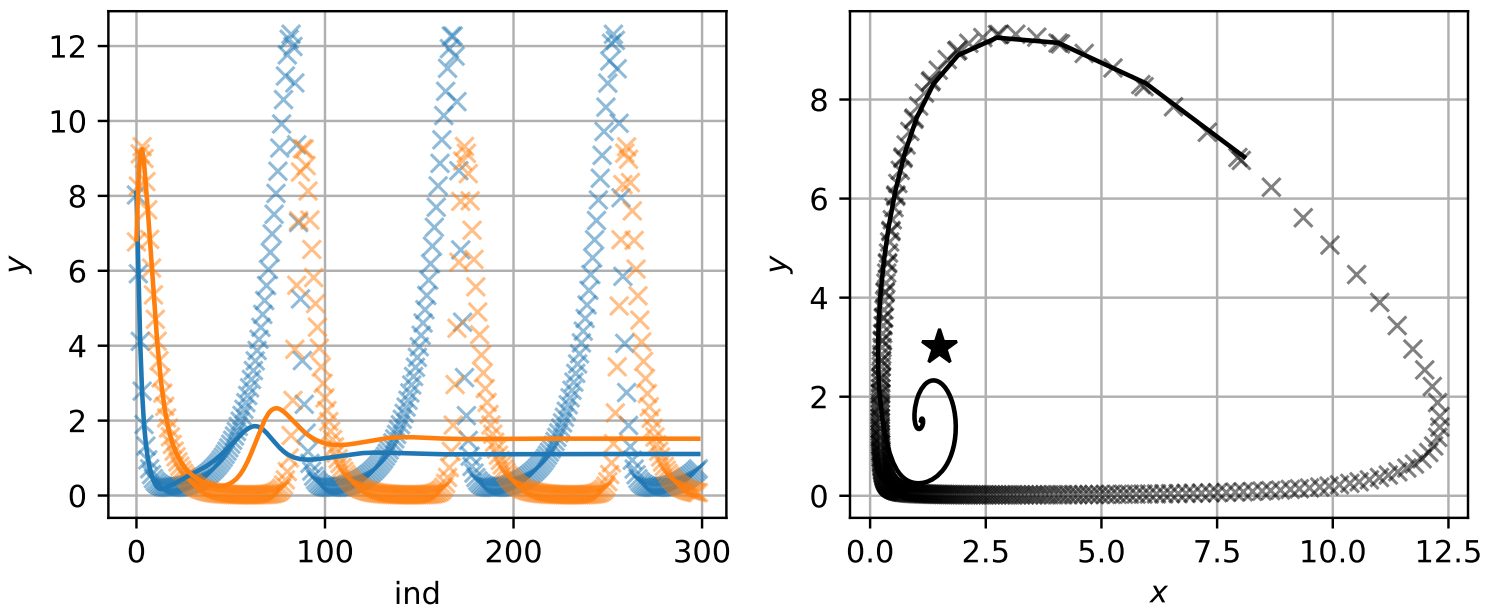

Figure: Harder sequential one-step test. Actual data are markers, and RNN predictions are given as lines.

RNNs: Lotka-Volterra

- longer sequence?

  $\rightarrow$ some memory effect, seems to recover amplitude after a while

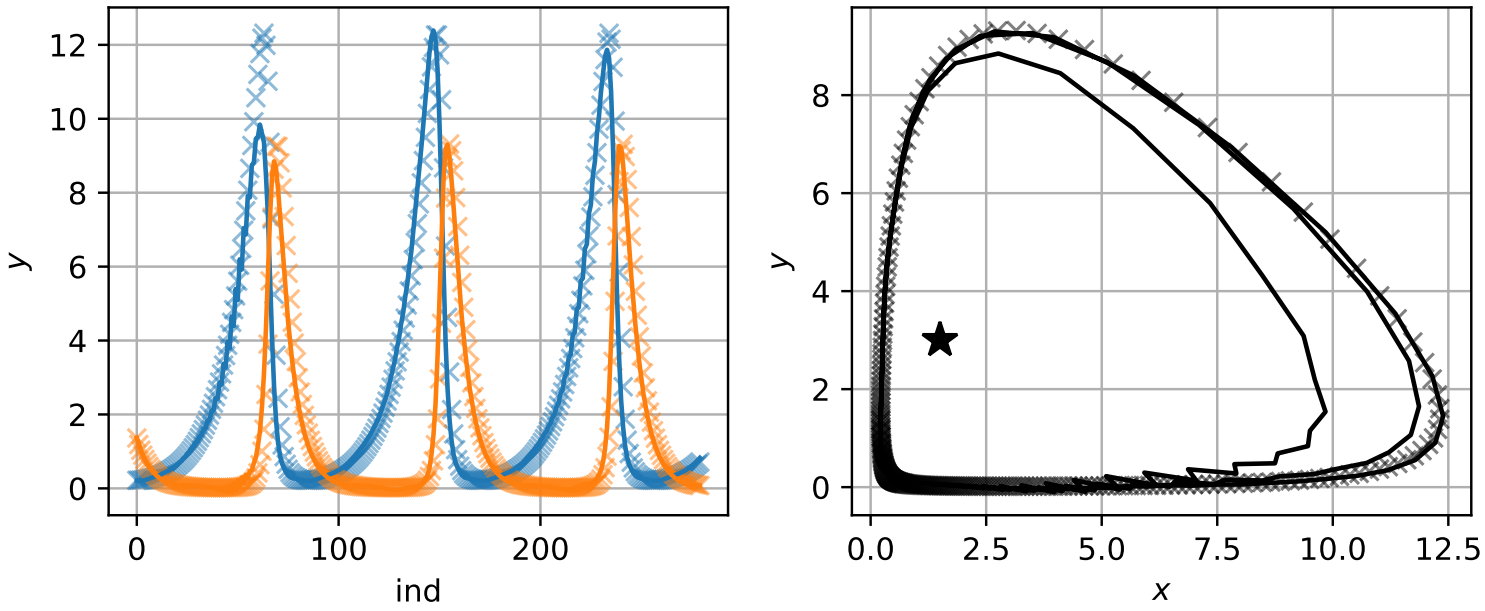

Figure: Harder sequential test with RNN trained on a sequence of twenty time-steps. Actual data are markers, and RNN predictions are given as lines.

RNNs: Lotka-Volterra

- example of an RNN doing insane things
  → **extinction** followed by **resurrection**!?
  → amplitude otherwise ok, phase errors though

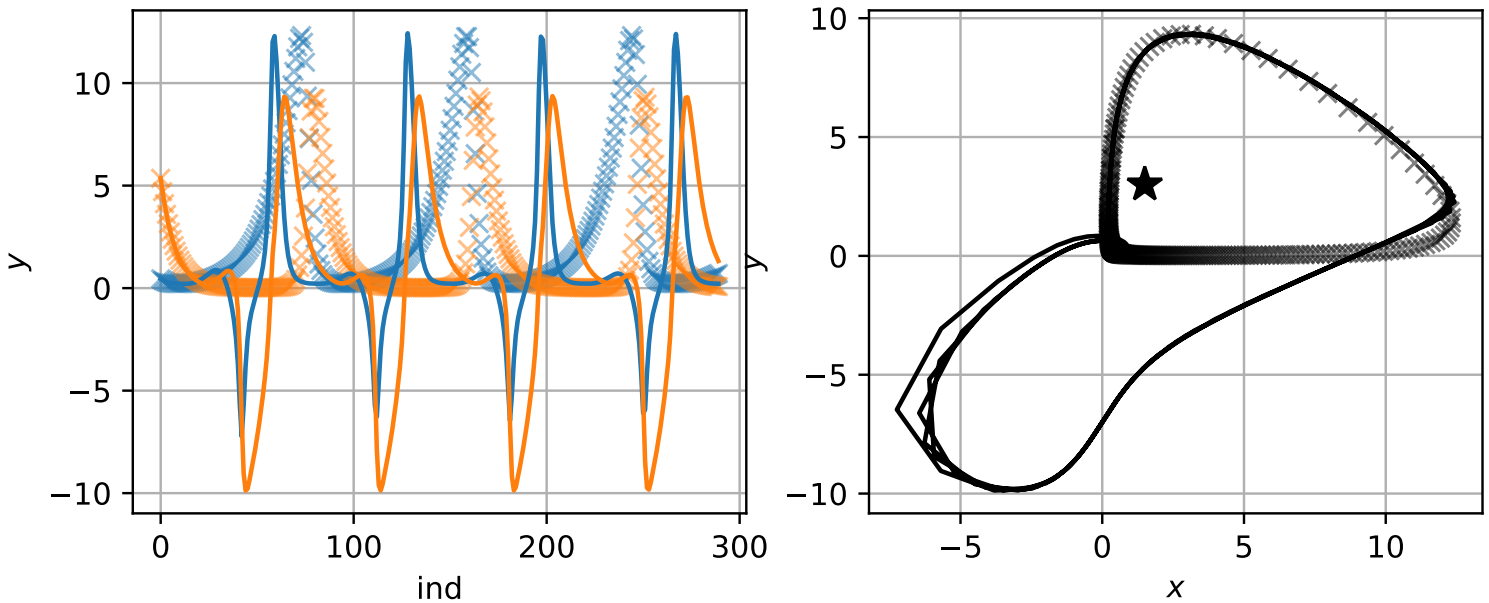

Figure: Harder sequential test with RNN trained on a sequence of ten time-steps. Actual data are markers, and RNN predictions are given as lines.

Demonstration

- auto-encoders

  → deeper + wider? data size?

  → **convolutional**?

- RNNs

  → needs to be constrained accordingly? (not trivial...)

  → more important to get **statistics** than exact states? (cf. weather vs. climate)

  → LSTMs and GRUs?

  → ConvLSTMs?

Moving to a Jupyter notebook →

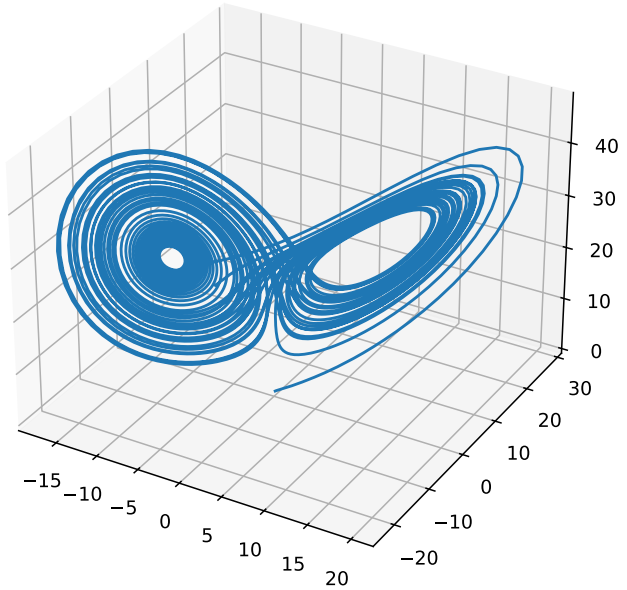

Figure: Butterfly diagram from a realisation of the Lorenz (1963) model. Example of a model with **chaos**, one feature of which is sensitive dependence on initial conditions.